Key User Flows

The five flows that make the platform work end-to-end.

① Agency Onboarding

STEPS

1. **Agency registers**
   Agency admin enters company name, Companies House number, email, password. Unique subdomain is auto-suggested from name.
   INSERT tenants (status=pending)
   Verification email

2. **Email verified**
   Admin clicks link in email. Token validated, account confirmed.
   UPDATE tenants (status=email_verified)

3. **Platform admin approves**
   Platform admin reviews and approves the agency via /admin/agencies. Manual step for v1 — keeps quality high at low volume.
   UPDATE tenants (status=active)
   Welcome email to agency

4. **Agency sets up branches + invites agents**
   Agency admin creates branches, invites agents by email. Agents receive invite link, set password, and land on their dashboard.
   INSERT branches, users Invite emails

5. **First listing created**
   Agent is prompted to add their first property. Empty state guides them through the listing form.

TENANT STATUS STATE MACHINE

pending — registered, awaiting email verification

↓ email verified

email_verified — awaiting platform admin approval

↓ approved by admin

active — fully operational

↓ (if needed)

suspended — admin action; listings hidden from search

② Listing Creation + Availability Slots

1. **Agent fills multi-step listing form**
   Address (autocomplete via OS Places API) → property type/bedrooms/price → description → features. HTMX validates each step inline without full page reload.
   hx-post per step INSERT properties (status=draft)

2. **Images uploaded**
   Agent selects images. JS requests pre-signed URLs from server, uploads directly to object storage, then confirms each upload. Server records storage_key in property_media. Drag-to-reorder sets sort_order.

Pre-signed PUT → S3/GCS INSERT property_media HTMX confirms each upload

3 **Agent adds viewing slots**

Simple slot picker: choose date, start time, duration (30/45/60 min), capacity (1 = private viewing, N = open house). Can add multiple slots. No external calendar sync in MVP.

INSERT availability_slots Slot list refreshes inline

4 **Agent publishes**

Status changes draft → active. Property appears in public search. Agent sees listing performance (views, saves, bookings) on their dashboard.

UPDATE properties (status=active)

③ Seeker: Search → View → Book (the core loop)

1 **Seeker searches**

Location (postcode or draw polygon on map), price range, bedrooms, furnished/pets etc. Results update via HTMX as filters change — no page reload, no spinner.

hx-get /properties with query params Full-text + PostGIS spatial query

2 **Seeker opens property detail**

Photos, description, features, neighbourhood summary. Available viewing slots shown prominently. If no slots, "message agent" is the CTA.

3 **Seeker picks a slot and books**

Clicks a slot → inline form appears (HTMX swap): name, email, phone. One click to confirm. No account required. Slot booked_count incremented atomically in a transaction to prevent double-booking.

hx-post /properties/:id/book INSERT bookings + UPDATE slot (atomic tx) Confirmation email to seeker + agent

4 **Confirmation page + account prompt**

Seeker lands on a confirmation page. If they don't have an account, a prompt: "Save this booking and get alerts for similar properties — takes 10 seconds." One click using the email already captured.

confirm_token in email link Optional account creation

5 **Automated reminders**

Background worker sends reminder emails 24h and 1h before the slot. Agent and seeker both notified. Easy reschedule/cancel link in email.

24h + 1h reminders Background job queue

6 **Agent marks outcome**

After the viewing, agent marks booking as attended or no_show from their diary. This feeds the analytics funnel and the agent's no-show rate stat.

UPDATE bookings (status=attended|no_show)

④ Secure Messaging

1 **Seeker sends enquiry**

From property detail page — seeker must be logged in to message (prevents spam). Creates a

INBOX DESIGN NOTES

conversation thread scoped to property + seeker + agent.

INSERT messages Email notification to agent

2 **Agent replies from inbox**

Agent sees all conversations in /dashboard/messages. HTMX polls for new messages every 30s. Reply is sent inline — no page navigation.

hx-get poll every 30s Email to seeker

3 **Thread continues**

Both sides can see full thread history. Agent can link to a booking slot directly from a message. Seeker can book without leaving the conversation.

- Conversations are grouped by property, not by seeker — so agents see all interest in a property in one view.
- Unread badge on nav. Response time is tracked and shown on agent profile (drives accountability).
- No WebSockets in MVP — HTMX polling every 30s is sufficient and simpler to deploy.
- Message content is stored in PostgreSQL (tenant-isolated). No PII in object storage.

⑤ Agent Analytics — The ROI Loop

👁️ **Listing Views**
How many seekers saw this listing

→

📅 **Bookings Made**
% who booked a slot

→

✅ **Attended**
% who showed up

Agent dashboard shows this funnel per property and across their whole portfolio. The headline metric is **attendance rate** — this is the number that proves platform ROI to agents, and the number that differentiates from portals who only show impressions.

Additionally: **avg. response time** (from first message to first reply), **no-show rate**, **saves per listing**, and **bookings per slot**. Platform admin sees these aggregated across all tenants.

⚡ HTMX Interaction Patterns

Interaction	HTMX trigger	Target	Server returns
Search filter change	`hx-get` on input change (debounced 300ms)	`#results-list`	Rendered property cards partial
Book a slot	`hx-post` on slot click	`#booking-panel`	Booking form or success confirmation
Table sort/filter	`hx-get` on column header / filter input	`#table-body`	Rendered `<tbody>` rows only
Message inbox poll	`hx-trigger="every 30s"`	`#message-list`	Updated thread list partial

Interaction	HTMX trigger	Target	Server returns
Send message	`hx-post` on form submit	`#message-list`	Appended new message bubble
Listing form steps	`hx-post` per step with validation	`#step-content`	Next step or inline errors
Image upload confirm	`hx-post` after S3 PUT completes	`#media-grid`	Updated image grid with new thumb
Mark booking outcome	`hx-patch` on attended/no-show button	Booking row	Updated row with new status badge